Introduction to Data structure and Algorithm

What is Data Structure

We can define data structures as follows:

It is a specialized format for organizing, processing, retrieving, and storing data in a computer's memory or disk to ensure efficient access and manipulation. It defines the logical relationships between data elements, enabling effective management for complex tasks.

Various and advanced types of data structures are used in almost every program or software system. Each data structure has their characteristics, features, applications, advantages, and disadvantages. So we must have good knowledge of data structures and we should know:

- **How many types of data structures are there and what are they used for?**
- **How do you identify and chose a *data structure* that is suitable for your a particular task?**

1. How Data Structure varies from Data Type:

We learned about the meaning of data structures. But some people confuse the concept of data types with data structures. So let's see some differences between them:

Data Type	Data Structure
The data type is the form of a variable to which a value can be assigned. For example integer, float, double, etc.	Data structure is a collection of different kinds of data. That entire data can be represented using an object and can be used throughout the program. For example, stack, queue, tree, etc.
It can hold value but not data. Therefore, it is data less.	It can hold multiple types of data within a single object.
There is no time complexity in the case of data types.	In data structure objects, time complexity plays an important role.

2. Classification of Data Structure:

Data structure **has many different uses in our daily life. There are many different data structures that are used to solve different mathematical and logical problems. By using *data structure*, one can organize and process a very large amount of data in a relatively short period. Let's look at different data structures that are used in different situations, Figure (1) represent the classification of data structure:**

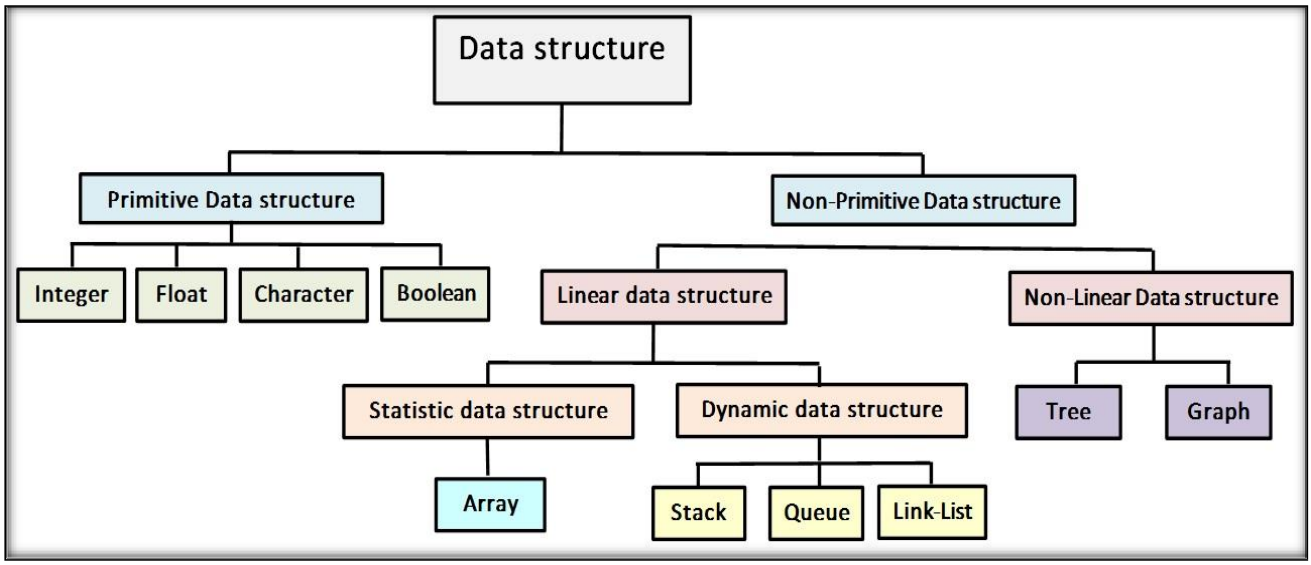

Figure (1): Classification of data structure

1. Primitive Data Structures

Primitive data structures **are those that are predefined by the programming language and cannot be broken down into simpler structures. They typically represent single values, such as numbers or characters. The following are common types of primitive data structures:**

1. ***Integer*****: It is used to represent whole numbers (positive or negative).**
2. ***Float*****: It is used to represent numbers with decimals or floating-point numbers.**
3. ***Character*****: It represents individual characters, usually stored as a single byte.**
4. ***Boolean*****: It represents a value of either true or false.**
5. ***String*****: It represents a sequence of characters.**

2. Non - Primitive Data Structure

Non-primitive data structures **are more complex and can store multiple values, including instances of primitive data structures. They are created using primitive data structures and can hold collections of data. Common types of non-primitive data structures include:**

1. Linear data structure

Linear Data Structures **are a type of data structure in computer science where data elements are arranged *sequentially* or *linearly*. Each is attached to its *previous* and *next* adjacent (مجاور), except for the *first* and *last* elements. *linear data structures include:***

a) *Static data structure:*

Static data structure has a fixed memory size. It is easier to access the elements in a static data structure. *An example of this data structure is an Array* which is a collection of elements stored in contiguous memory locations.

b) *Dynamic data structure:*

In the dynamic data structure, the *size is not fixed*. It can be *randomly* updated during the runtime which may be considered *efficient concerning* the memory (space) complexity of the code. *Examples of this data structure are :*

i. ***Stacks:*** **It follows the Last-In-First-Out (LIFO) principle for data storing and retrieving data.**

ii. ***Queues:*** **It follows the First-In-First-Out (FIFO) principle for storing and retrieving data.**

iii. ***Linked Lists:*** **A linear collection where each element (node) contains a reference (link) to the next node in the sequence.**

2. Non-linear data structure:

Data structures where data elements are not placed sequentially or linearly are called non-linear data structures. In this type, we can't traverse all the elements in a single run only. *Examples of non-linear data structures are:*

i. ***Trees:*** **Hierarchical structures consisting of nodes that represent a relationship between parent and child nodes.**

ii. ***Graphs:*** **A collection of nodes (vertices) and edges that represent relationships between them.**

Seq.	Linear Data Structure	Non-linear Data Structure
1.	In a linear data structure, data elements are arranged in a linear order where each and every element is attached to its previous and next adjacent except the first and last element.	In a non-linear data structure, data elements are attached in hierarchically manner. يتم الربط بأسلوب هرمي
2.	In linear data structure, single level is involved.	Whereas in non-linear data structure, multiple levels are involved.
3.	Its implementation is easy in comparison to non-linear data structure.	While its implementation is complex in comparison to linear data structure.
4.	In linear data structure, data elements can be traversed in a single run only.	While in non-linear data structure, data elements can't be traversed in a single run only.
5.	In a linear data structure, memory is not utilized in an efficient way.	While in a non-linear data structure, memory is utilized in an efficient way.
6.	Its examples are: array, stack, queue, linked list, etc.	While its examples are: trees and graphs.
7.	Applications of linear data structures are mainly in application software development.	Applications of non-linear data structures are in Artificial Intelligence and image processing.
8.	Linear data structures are useful for simple data storage and manipulation.	Non-linear data structures are useful for representing complex relationships and data hierarchies, such as in social networks, file systems, or computer networks.
9.	Performance is usually good for simple operations like adding or removing at the ends, but slower for operations like searching or removing elements in the middle.	Performance can vary depending on the structure and the operation, but can be optimized for specific operations.

In the upcoming lectures, we will begin to learn about the types of data structures in terms of their properties, importance, and how to use them.